Numerical Simulation of a Damped Spring-Mass System: Accuracy, Stability and Performance of Euler and RK4

Samuel Davis 36136603 | MMA3001 Numerical Methods and Machine Learning, Individual Project

GitHub repository: https://github.com/Sammmydav/spring-damper-system

1. Engineering problem

Context. Spring-mass-damper systems are the basic model for vibration in mechanical and aerospace engineering: a vehicle suspension, a vibration isolator under an instrument, a landing-gear strut. This project uses the quarter-car suspension as the motivating example, where one quarter of the body mass is supported by a spring and a viscous damper. Engineers simulate these systems to predict how quickly a disturbance decays and whether the response rings or overshoots.

Problem. The free response of a linear single-degree-of-freedom oscillator, m x'' + c x' + k x = 0 with x(0) = x0 and x'(0) = v0, is solved numerically. The question addressed is which time-stepping method and which time step should be used, and how that choice can be justified by evidence rather than preference. A method that is too inaccurate gives wrong predictions, and an unstable one can produce a response that grows without bound even though the real system loses energy.

Why it matters. Larger simulations of the same type (multi-body, finite element) rely on the same integrators. Understanding accuracy, stability and cost on a problem with an exact answer makes the trade-offs measurable.

Intended use. The code is a validated reference implementation for choosing an integration method and step size for linear one-degree-of-freedom vibration problems.

Scope and limitations. Linear spring, viscous damping, no external forcing, one degree of freedom, fixed time step. Nonlinear springs, Coulomb friction, forced vibration and multi-body systems are out of scope.

2. Inputs, outputs and data

Inputs.

Input	Symbol	Unit	Valid domain
Mass	m	kg	finite, > 0
Damping coefficient	c	N s/m	finite, >= 0
Stiffness	k	N/m	finite, > 0
Initial displacement / velocity	x0, v0	m, m/s	finite
Time step	dt	s	finite, > 0

Input	Symbol	Unit	Valid domain
End time	T	s	finite, >= dt
Method	method	n/a	"euler" or "rk4"

Outputs. Arrays of time t [s], displacement x [m] and velocity v [m/s]; error metrics against the exact solution (maximum absolute error and RMSE); the observed convergence order; the explicit-Euler stability limit [s]; and runtime [s]. The displacement and velocity histories are the response an engineer inspects for overshoot, settling time and decay; the error metrics quantify solver accuracy against the exact solution; the observed order and the Euler stability limit guide the choice of dt; and runtime supports cost comparisons.

Invalid input handling. Non-numeric, non-finite or out-of-domain values (for example m <= 0, c < 0, dt <= 0) and unknown method names raise a ValueError with a descriptive message. Each case is covered by an automated test.

Data. No external dataset is used. The inputs are synthetic parameter sets, and results are checked against mathematical ground truth. Two kinds of cases are used: (a) normalised cases (m = 1 kg, k = 4 N/m) covering all four damping regimes, and (b) two cases with realistic scales whose parameters were chosen inside published ranges: a quarter-car suspension (m = 300 kg, k = 16,000 N/m, c = 1,000 N s/m, damping ratio 0.23), using ranges reported by Ugalde Garduño et al. (2026), and a lightly damped steel specimen (m = 1 kg, damping ratio 0.005), following the range reported by Acharya et al. (2026). The specimen stiffness (1,000 N/m) is an illustrative choice not taken from the source. These are representative values, not measurements, so the realistic cases test the solvers at realistic scales; they do not validate the physical model against real data. No personal or proprietary data are used, and the parameter ranges come from the cited published sources.

3. Computational approach

Formulation. The second-order equation is rewritten as a first-order system for y = [x, v]: dy/dt = [v, -(c v + k x)/m], following the unit notes (Atkinson, 2026c).

Methods.

- *Explicit (forward) Euler* (baseline): y(n+1) = y(n) + dt f(y(n)). One right-hand-side evaluation per step; first-order accurate (Atkinson, 2026a).
- *Classical Runge-Kutta (RK4)* (selected): four evaluations per step combined with weights 1, 2, 2, 1 divided by 6; fourth-order accurate (Atkinson, 2026b).

Why these methods. Euler is the simplest possible baseline and is the natural method to test for stability problems. RK4 is the standard higher-order explicit method and gives a clear accuracy-versus-cost comparison against it. Both are explicit, which keeps the implementation transparent and makes the stability analysis tractable.

Exact solution. The analytical solution is used for validation. With natural frequency wn = sqrt(k/m) and damping ratio zeta = c / (2 sqrt(k m)), there are four cases: undamped and underdamped (zeta < 1, decaying cosine and sine), critically damped (zeta = 1, an exponential times a linear term) and overdamped (zeta > 1, two real exponentials). The code selects the case from zeta; the damping regimes

and the underdamped solution follow standard texts (Atkinson, 2026c; Moebs et al., 2016, Section 15.5), and the critically damped and overdamped forms follow from the roots of the characteristic equation m r^2 + c r + k = 0 (repeated and real distinct roots respectively).

Settings and assumptions. Fixed time step; the grid has n = round(T/dt) steps; the history of x and v is stored. The main test case is m = 1 kg, c = 0.4 N s/m, k = 4 N/m, x0 = 1 m, v0 = 0 (zeta = 0.1, wn = 2 rad/s), simulated to T = 5 s.

Implementation. Python with NumPy (Harris et al., 2020), SciPy (Virtanen et al., 2020) and pytest; code is organised into modules for the model and validation (models), the exact solution (analytical), the solvers (solvers, fast, alternatives), analysis tools (analysis) and named cases (cases). Every function has a docstring describing parameters, returns and exceptions, and HTML documentation is generated with pdoc.

Known limitations. Fixed step size in the hand-written solvers (no error control; SciPy's adaptive solvers are compared in Section 4), and a linear one-degree-of-freedom model.

4. Alternative solutions

Alternative solutions considered. Besides the explicit Euler baseline and the selected RK4, three further approaches were compared: backward (implicit) Euler, which is unconditionally stable; symplectic (semi-implicit) Euler, designed to keep energy bounded in conservative systems; and SciPy's adaptive explicit Runge-Kutta solvers (RK45 and DOP853), the established method taught in the unit (Atkinson, 2026b; Virtanen et al., 2020). The exact solution is not a competing method, since it exists only for this linear, unforced case; it is used as ground truth. Higher-order implicit and multistep methods were not implemented because the problem is neither stiff nor large.

Method	Accuracy (main case, T = 5 s)	Stability and energy behaviour	Cost and complexity
Explicit Euler (baseline)	1st order; max error 3.7e-3 m at dt = 0.001 s	Conditionally stable (dt <= 2 zeta/wn); undamped energy +639% after 50 s	1 right-hand-side (RHS) evaluation per step (5,000 here); simplest
Backward Euler	1st order; max error 3.7e-3 m at dt = 0.001 s	Unconditionally stable but strongly dissipative; undamped energy -86% after 50 s	One 2 x 2 solve per step (closed form here); a linear solve per step in general
Symplectic Euler	1st order; max error 9.9e-4 m at dt = 0.001 s	Energy stays bounded when undamped (+0.9% after 50 s)	Same cost as Euler; suited to conservative systems
RK4 (selected)	4th order; max error 3.0e-6 m at dt = 0.05 s	Conditionally stable; undamped relative energy change -4.4e-9 after 50 s	4 RHS evaluations per step (400 here); easy to implement
SciPy RK45 / DOP853	Adaptive; 5.4e-7 m (rtol 1e-6, 362 evaluations) and 1.1e-9 m (rtol 1e-9, 404 evaluations)	Error control chooses the step; no stability limit to tune by hand	Library call; slower in wall-clock time here (per-call overhead); black box

Table 1. Comparison of the integrators considered. Accuracy and cost refer to the main case (zeta = 0.1, T = 5 s); energy figures refer to the undamped oscillator at dt = 0.01 s. Cost is counted in RHS evaluations, which does not depend on the machine.

Robustness. For the lightly damped specimen at dt = 0.001 s the maximum displacement error relative to the initial displacement was 5.0 for explicit Euler (unstable), 0.48 for backward Euler, 0.017 for symplectic Euler and 6.0e-7 for RK4. Backward Euler is stable but its numerical damping removes the oscillation far faster than the physics does, so stability alone is not enough; neither first-order explicit nor implicit Euler suits lightly damped or undamped systems.

Why RK4 was selected. RK4 reached 3.0e-6 m with 400 right-hand-side evaluations, whereas the first-order methods needed 5,000 to reach about 1e-3 m. SciPy's adaptive solvers used a similar number of evaluations (362 and 404) and give error control without choosing dt by hand, so they would be the safer default in production. RK4 was developed here because it is transparent, fast for a problem this size, and exposes the step-size, stability and cost trade-offs the project sets out to measure.

Memory and scalability. All fixed-step methods store the same history (16 bytes per step), so memory does not distinguish them. For larger systems RK4's four evaluations per step and the per-step linear solve of implicit methods would matter more; with two states this was reasoned about rather than measured.

5. Validation

What is validated. The correctness of the numerical solvers: that they implement the intended mathematics and converge at the expected rates. This is verification of the implementation, not validation of the physical model against measurements.

Benchmarks and why they are appropriate. (i) The exact analytical solution, which is the ground truth for this equation; (ii) SciPy's solve_ivp with the DOP853 method and tight tolerances (Hairer et al., 1993; Virtanen et al., 2020), an independent established computational method, which guards against an error shared by my analytical code and my solvers; (iii) physical constraints: energy is conserved for the undamped system and decreases monotonically when damping is present.

Acceptance criteria (set before running the studies).

- RK4 maximum displacement error below 1e-6 m on the main case at dt = 0.01 s.
- Observed convergence order within 0.15 of 1 for Euler and within 0.5 of 4 for RK4 (checked at fine steps).
- RK4 agrees with solve_ivp to better than 1e-7 m.
- Energy of the exact damped solution is non-increasing; the exact undamped energy is constant to round-off.

Results across the four damping regimes (dt = 0.01 s, T = 5 s, m = 1 kg, k = 4 N/m, x0 = 1 m, v0 = 0.5 m/s).

Regime	Euler max error [m]	Euler RMSE [m]	RK4 max error [m]	RK4 RMSE [m]
Undamped (zeta = 0)	1.05e-1	4.60e-2	1.12e-8	5.32e-9
Underdamped (zeta = 0.1)	4.09e-2	2.27e-2	5.06e-9	2.73e-9

Regime	Euler max error [m]	Euler RMSE [m]	RK4 max error [m]	RK4 RMSE [m]
Critical (zeta = 1)	3.44e-3	1.83e-3	2.08e-9	7.63e-10
Overdamped (zeta = 2.5)	1.75e-3	7.68e-4	2.80e-8	5.50e-9

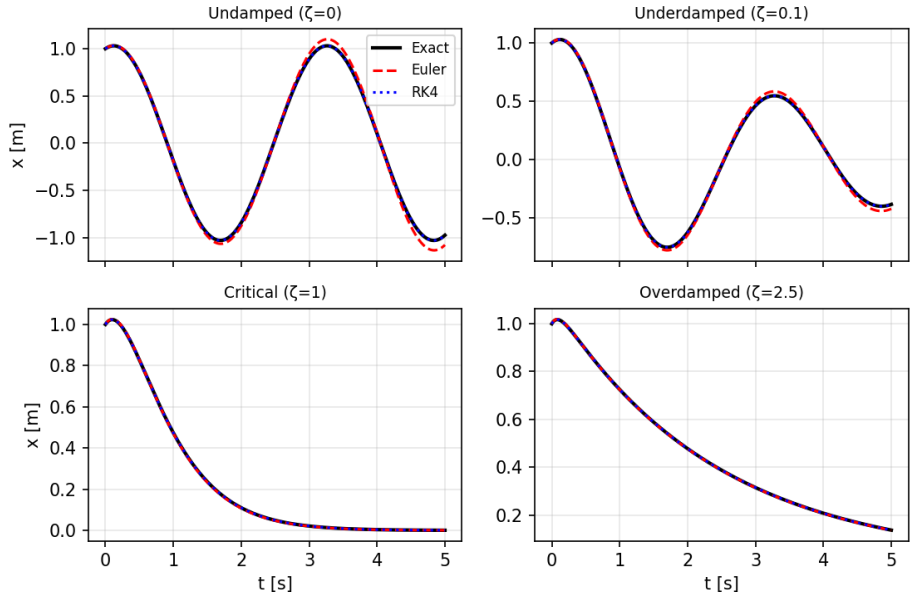


Figure 1. Numerical and exact displacement for the four damping regimes (dt = 0.01 s).

RK4 meets the 1e-6 m criterion in every regime. Euler is orders of magnitude less accurate and is worst for the undamped case, where its error accumulates over the oscillations. RK4 agrees with solve_ivp to 5.1e-9 m, and the exact solution agrees with solve_ivp to 8.9e-13 m, which also confirms the analytical code independently.

Convergence study. The time step was halved six times from 0.2 s on the main case. Euler's error fell from 2.18 m to 1.2e-2 m and RK4's from 7.8e-4 m to 4.5e-11 m. The observed orders approach the theoretical values: Euler 1.82, 1.38, 1.18, 1.09, 1.04, 1.02 and RK4 4.02, 4.01, 4.01, 4.00, 4.00, 4.00. The inflated first Euler values occur because dt = 0.2 s is above the Euler stability limit for this case (Section 6), so the asymptotic regime is only reached once the step is small enough.

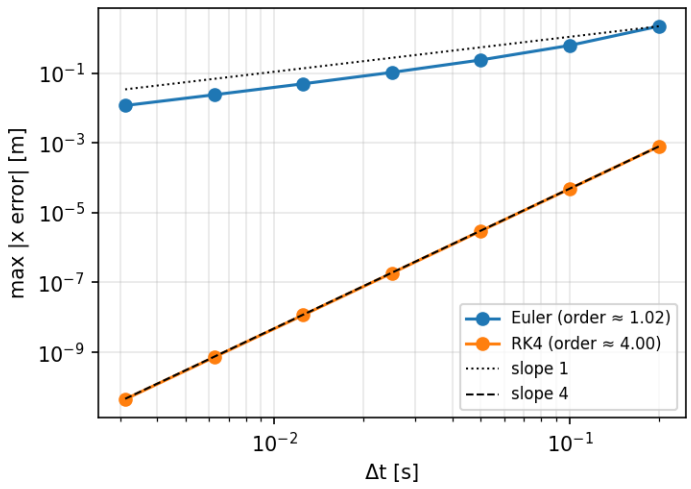


Figure 2. Maximum displacement error against time step on a log-log scale. Dotted and dashed lines show slopes 1 and 4.

Physical checks. For the undamped case (initial energy 2 J), RK4 held the energy at 2.0000 J after 20 s while Euler had grown to 4.45 J, an increase of about 120% (Figure B1, Appendix B). The exact damped solutions lose energy monotonically in all tested regimes.

Automated verification. A pytest suite of 84 tests checks input validation, regime classification, the exact solution (initial conditions, residual of the ODE using finite differences, energy behaviour), agreement of both solvers with the exact and SciPy solutions, the convergence orders, the Euler stability limit either side of its value, the realistic cases, that the optimised solver matches the reference one, and the alternative integrators (convergence order, energy behaviour and stability). The generated test report is in the repository.

What the validation cannot establish. That the linear model represents a real suspension or specimen (nonlinear springs, dry friction, forcing and measurement noise are not tested); that the chosen parameters are accurate for any real vehicle; and behaviour outside the tested parameter ranges. Because the benchmark is an exact solution of the same equation, the validation says nothing about modelling error.

Did validation influence method selection? Yes. The convergence and error results provided the evidence for choosing RK4 over Euler, and the stability test determined the step sizes at which Euler is usable.

6. Comparison, optimisation and performance

Baseline against selected method. At the same step RK4 is far more accurate (validation table, Section 5), but each step costs more. Because the costs differ, the fair comparison is error against runtime rather than error against step size.

Stability. For explicit Euler each eigenmode is multiplied by (1 + lambda dt) per step (the amplification-factor argument in Atkinson, 2026a, applied here to the complex eigenvalues of the oscillator), so stability requires |1 + lambda dt| <= 1 for both eigenvalues of the system matrix. This gives dt <= -2

Re(lambda) / |lambda|^2, which equals 2 zeta / wn for an underdamped system. For the main case this limit is 0.1 s. Numerically, Euler's energy decays for dt = 0.9 times the limit and grows for dt = 1.1 times the limit, confirming the analysis. For the undamped system the limit is zero: Euler amplifies the solution by a factor of sqrt(1 + (wn dt)^2) per step for every dt > 0.

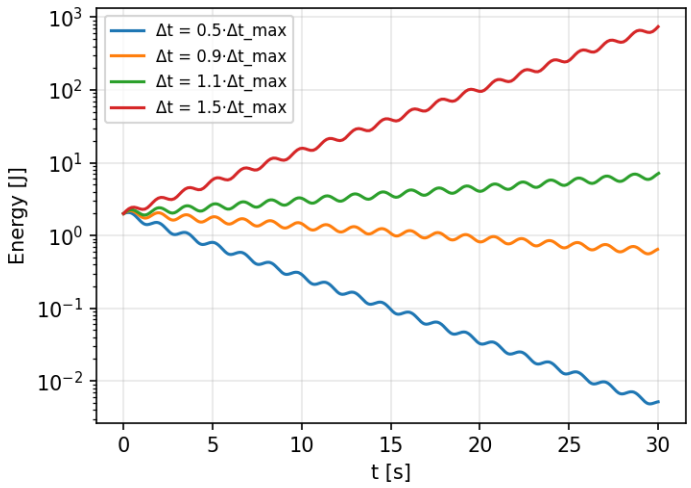


Figure 3. Euler energy history for time steps either side of the analytical stability limit (zeta = 0.1).

Realistic cases. The stability limit depends strongly on damping. For the quarter-car case (zeta = 0.23) the limit is 0.0625 s; Euler is stable at dt = 0.01 s but its error is still 6.3% of the initial displacement, against 4.0e-7 for RK4. For the lightly damped specimen (zeta = 0.005) the limit shrinks to 3.2e-4 s, so Euler is unstable at dt = 1e-2 s and 1e-3 s (errors of order 10^9 and 5 times the initial displacement), and at dt = 1e-4 s, which is stable, it is still 13% in error. RK4 gives relative errors of 5.9e-3, 6.0e-7 and 6.0e-11 at the same steps. Lightly damped systems are therefore where an unstable baseline fails most visibly (Figure B2, Appendix B, shows both cases). Together with the convergence study, which varies dt, and the three damping cases, which vary zeta, this forms the sensitivity analysis of the solution to its two controlling parameters.

Accuracy per unit cost. Runtimes are best-of-several measurements on a HP Pavillion, 12th Gen Intel(R) Core(TM) i5-1235U (1.30 GHz) and Python 3.10.11; timings vary between machines and between runs. A single RK4 step costs about 4.8 times as much as a single Euler step (17.5 microseconds against 3.6 microseconds per step at dt = 0.001 s, NumPy solvers), but RK4 can use a much larger step for the same error, which more than compensates. RK4 reached a maximum error of 1e-6 m in 2.47 ms (interpolated within the tested range). Euler never gets close to that error in the tested range: its smallest error was about 3e-3 m at dt = 7.8e-4 s, and its error falls only in proportion to dt (about 3.7 times dt). Reaching 1e-6 m would need dt of roughly 2.7e-7 s, about 1.9e7 steps. The runtime is therefore extrapolated from the observed first-order trend rather than measured: about 93 s with the NumPy solver (the notebook's extrapolation, roughly 5 microseconds per step at the finest steps), which makes RK4 about 38,000 times cheaper for this accuracy. At a looser target of 1e-3 m, which Euler does reach in the tested range, Euler needed 77.1 ms against 0.41 ms for RK4 (the RK4 value extrapolated from its coarsest steps), a ratio of about 190.

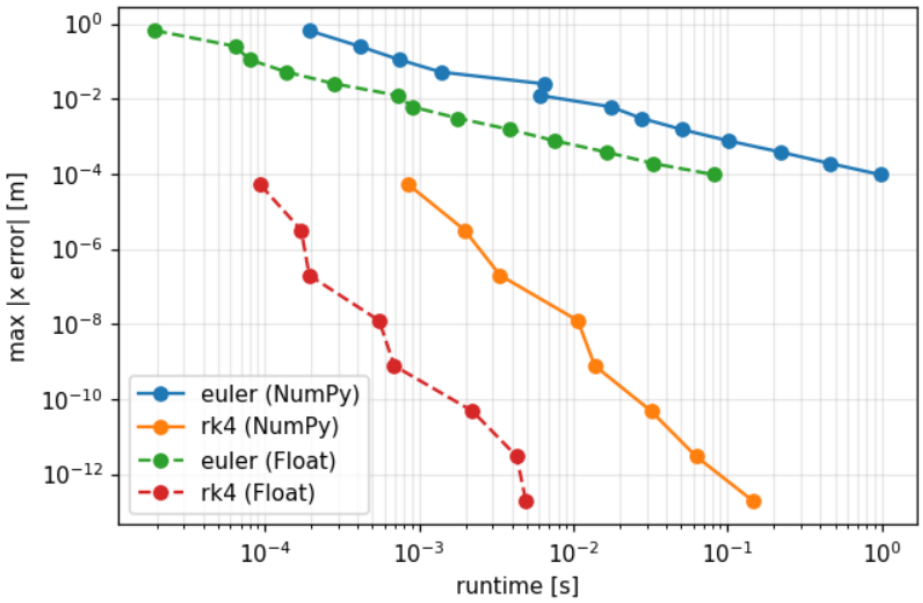


Figure 4. Error against runtime for Euler and RK4, using the NumPy and the pure-float solvers. Euler is run at much finer steps to reach small errors. Points at the finest steps use a single timing run, so they are noisier.

Profiling and performance estimates. A cProfile run of RK4 (5,000 steps) attributed almost all the time to rk4_step and its helper that evaluates the right-hand side, including the creation of 20,000 small NumPy arrays; profiler overhead inflates the absolute figures, but the distribution is informative. A hand count gives roughly 9 floating-point operations per Euler step and 40 to 50 per RK4 step [hand estimate, check before citing]. The state has two doubles (16 bytes), and the stored history grows as (n + 1) x 16 bytes, which is about 80 kilobytes for 5,000 steps. With roughly 40 to 50 floating-point operations and about 16 bytes of history stored per RK4 step, the arithmetic intensity is of the order of 3 FLOP per byte (about 0.5 for Euler, at about 9 operations per step; hand estimates). Both are small, so the load is light and the measured runtime is dominated by Python and NumPy call overhead, not by floating-point work.

Optimisation. The profile motivated a second implementation, which keeps x and v as plain floats and writes the Euler and RK4 updates out explicitly (module fast). It implements the same mathematics: results agree with the NumPy version to better than 1e-12 (verified by tests). The pure-float version is 13.0 times faster for Euler and 24.1 times faster for RK4 at dt = 0.001 s (Euler 18.08 ms against 1.39 ms, RK4 87.68 ms against 3.64 ms, for 5,000 steps), or about 0.28 and 0.73 microseconds per step. Because RK4 gains more, the RK4-to-Euler cost ratio per step falls from about 4.8 to about 2.6, still below the hand-counted ratio of about 5, which suggests that loop and indexing overhead in Python still contributes. To reach 1e-6 m, RK4 with the float solver needs 0.18 ms, while extrapolated Euler would need about 7.8 s, so the selected method is about 43,000 times cheaper than the baseline. At the looser 1e-3 m target the float solvers need 5.8 ms (Euler) against 0.045 ms (RK4, extrapolated), a ratio of about 130.

Final design and trade-offs. RK4 with the pure-float implementation is selected: it meets all acceptance criteria, is the fastest in wall-clock time to a given accuracy of the methods compared (SciPy's adaptive

solvers need a similar number of right-hand-side evaluations but carry more per-call overhead), and is stable for all tested cases at the steps used. Its costs are a more complex implementation and four evaluations per step. Euler is not recommended for lightly damped or undamped systems, and where it is used the step must stay well below 2 zeta / wn.

7. Limitations and lessons learned

Limitations. (1) The model is linear with one degree of freedom, so conclusions do not carry over to nonlinear or forced systems without further testing. (2) Only a small set of alternatives was compared (three Euler variants, RK4 and SciPy's adaptive solvers); higher-order implicit or multistep methods could change the recommendation, particularly for stiff systems. (3) The realistic cases use representative parameters, not measured data, so the model itself is not validated against experiment. (4) Timing results depend on the machine and on Python overhead, and should be read as relative.

Lessons learned. (1) Verifying convergence order on a problem with an exact answer is a powerful check of both the code and the theory. (2) Stability limits depend on the physics (damping), not just on the method, and the worst case can be a lightly damped system. (3) Profiling before optimising showed that the real bottleneck was overhead, not arithmetic, and changed which optimisation was worth doing. (4) Comparing methods by accuracy per unit cost leads to a different conclusion than comparing them at equal step size.

Engineering interpretation. For the quarter-car suspension (natural frequency about 1.16 Hz, period about 0.86 s), RK4 at dt = 0.01 s has a maximum error of about 4e-7 of the initial displacement, while explicit Euler at the same step is 6.3% in error. A ride simulation should therefore use RK4 (or an adaptive solver) with a step of 0.01 s or smaller, about 86 steps per oscillation period, and any explicit Euler step must stay below 2 zeta / wn, about 0.06 s for this suspension. For lightly damped structures (zeta of about 0.005) the Euler limit falls to about 3e-4 s, which makes it impractical.

Possible improvements. Add forcing and a nonlinear spring; compare higher-order implicit methods on stiffer systems; validate the model against a measured free-decay record, for example recorded with a phone accelerometer, by fitting the damping ratio and natural frequency.

8. AI use reflection

Two AI tools were used in this project. The main tool was Claude (Anthropic, Claude Sonnet 5.5), accessed through the claude.ai chat interface. I also used Google Gemini, through the Google search bar, for general queries, such as explaining concepts I found confusing, rather than for specific code fixes. Claude was used to explain the project brief, to suggest candidate project topics, and to generate the project scaffold: the solvers, the exact solution, the analysis functions, the tests, the notebook, the study scripts, the README, the report outline and the figures. I also used it to evaluate improvements I wanted to make, checking whether each sat within the limits of the brief, was feasible, and would improve the project overall. It proposed the pure-float optimisation, helped find published parameter ranges for the realistic cases, and explained how Git and GitHub work. Claude also produced the first draft of this report from the project results.

The AI contribution to the code was large. The initial code and tests were AI-generated, and I modified only a small part of the final code myself. The report was drafted by AI and then rewritten by me. The

README was also drafted by AI, with sections left for me to complete, and I filled in the sections it prompted me for, such as the engineering context, the key results and the AI declaration. I used AI because I am not yet skilled enough to code a numerical simulation of a damped spring-mass system on my own, and because I was unfamiliar with GitHub, including what it is before this unit, so help in those areas would greatly improve the quality of the project. I also work better when preparing reports and presentations if I have a detailed draft or scaffold to build on.

I did not treat AI output as correct by default. After every addition to the code I ran the full test suite locally and confirmed that it passed. I reproduced the convergence orders in the notebook, compared the results with SciPy's solve_ivp, and confirmed that the parameter ranges used for the realistic cases were correct. I also referred back to the brief throughout to make sure the project stayed within its requirements. The decisions that remained my responsibility were choosing the spring-mass-damper simulation as the topic from the options proposed, deciding to add realistic parameter cases rather than fit measured data, noticing that the report lacked a dedicated alternative-solutions section and asking for it to be added, choosing the report structure and APA referencing, supplying the unit notes as sources, and approving each addition to the project. Suggestions such as the damping cases and the acceptance thresholds came from Claude, and I reviewed and accepted them.

The AI also made errors and unhelpful suggestions. Its first suggested grouping of Git commits put a test file in a commit before the module it imports, which would have left that commit failing. It initially expressed more confidence that a simulation-only project satisfied the brief than was justified. The first version of its time-to-accuracy calculation silently clamped Euler's result to the edge of the tested range, because Euler never reaches 1e-6 m there, which understated RK4's advantage as about ten times instead of several orders of magnitude, so the calculation was corrected to flag extrapolation. Its timing figures were measured on a different machine from mine and did not apply to my computer, so I replaced them with my own measurements. Some of its reference suggestions also needed correcting: a learning resource it suggested turned out to be a practice question rather than a usable source, and author details had to be fixed after the source papers were checked. Its FLOP counts are hand estimates, and the parameter ranges came from search excerpts that had to be checked against the papers.

Working through this project improved my understanding of how Euler's method and RK4 solve ordinary differential equations, and showed me that these methods are needed to calculate things that affect everyday life, rather than being topics that are only nice to know. It taught me how to use GitHub and a repository, and how to test code to confirm that tests pass. I still find the coding itself difficult, but the concepts behind it make much more sense to me now.

9. References

Acharya, M., Mishra, S., Singh, B., Adhikari, R., Koirala, N., & Bhatt, M. R. (2026). Study on damping property and vibration behaviour of SDOF system. *Journal of Science and Engineering, 13*(1), 167–173. https://doi.org/10.3126/jsce.v13i1.89608

Atkinson, C. (2026a). *7.3 Euler's method* [Course notes]. MMA3001 Numerical Methods and Machine Learning, Monash University.

Atkinson, C. (2026b). *7.4 Runge-Kutta methods* [Course notes]. MMA3001 Numerical Methods and Machine Learning, Monash University.

Atkinson, C. (2026c). *7.5 Higher-order ordinary differential equations* [Course notes]. MMA3001 Numerical Methods and Machine Learning, Monash University.

Hairer, E., Nørsett, S. P., & Wanner, G. (1993). *Solving ordinary differential equations I: Nonstiff problems* (2nd rev. ed.). Springer.

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., . . . Oliphant, T. E. (2020). Array programming with NumPy. *Nature, 585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Moebs, W., Ling, S. J., & Sanny, J. (2016). *University physics volume 1*. OpenStax. https://openstax.org/books/university-physics-volume-1/pages/1-introduction

Ugalde Garduño, A. O., Ayala-Jaimes, G., & Cervantes Huerta, L. (2026). Modelling, stability analysis, and PID control of a quarter-car suspension system using frequency and state-space methods. In *Proceedings of the 13th International Conference of Control Dynamic Systems, and Robotics (CDSR 2026)* (Paper No. 151). Avestia. https://doi.org/10.11159/cdsr26.151

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., . . . SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods, 17*(3), 261–272. https://doi.org/10.1038/s41592-019-0686-2

Appendix A: Repository structure

src/ (models, analytical, solvers, fast, alternatives, analysis, cases); tests/ (pytest suite); scripts/run_study.py (convergence, stability, timing and figures); scripts/alternatives_study.py (comparison of alternative integrators); notebooks/analysis.ipynb (all experiments); docs/ (pdoc HTML in docs/api, test_report.html, report outline); README.md; LICENSE.

Appendix B: Additional figures

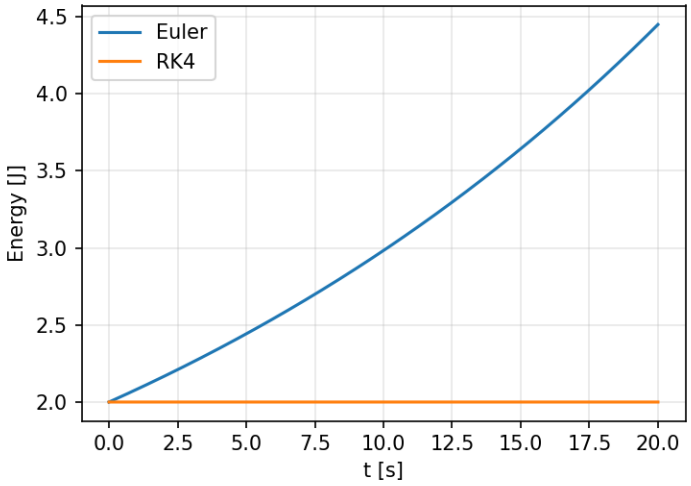


Figure B1. Energy of the undamped oscillator (dt = 0.01 s, exact value 2 J): Euler grows, RK4 stays constant.

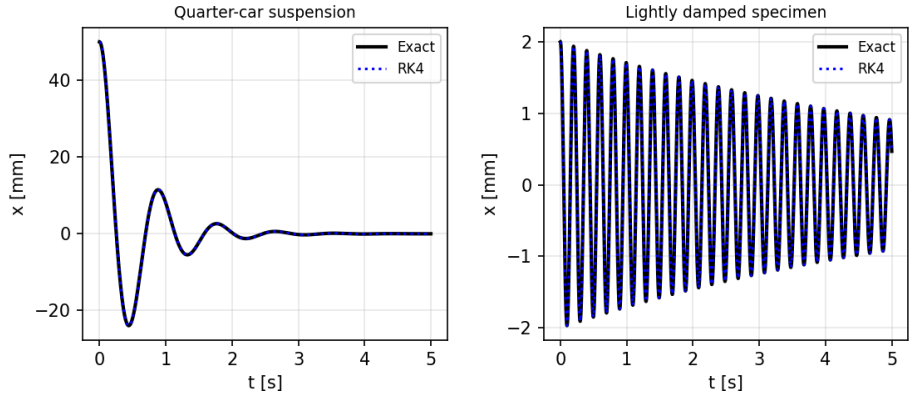


Figure B2. Exact and RK4 displacement for the two realistic cases.